

Three.js Strategy Guide for StoryStacker

Introduction

Purpose: explain how and why to use Three.js for StoryStacker's web-based Asset Lab, prototype stacking game.

Summary: Three.js is a JavaScript library that sits on top of WebGL. It gives us an approachable, production-proven solution.

Why Three.js for StoryStacker

- 1) Zero-install demos: anyone with a modern browser can view and interact with assets.
- 2) Fast iteration: single-file HTML prototypes; instant reload; easy to host on GitHub Pages.
- 3) Native GLB/gLTF support: loads your optimized Runtime assets directly.
- 4) Extensible UI: combine WebGL rendering with HTML/CSS panels for metadata and tools.
- 5) Long-term fit: the same viewer can evolve into a public demo and internal placement sandbox.
- 6) Cost and footprint: no engine runtime download; tiny site sizes (MBs, not GBs).

Comparison & Trade-offs

Three.js vs alternatives:

- Unity/Godot (Web export): powerful, but heavier downloads and build steps. Great for final game, less ideal for prototyping.
- Raw WebGL/WebGPU: maximum control, but slow to build from scratch. Three.js abstracts the hard parts.
- Model viewers (off-the-shelf): quick start but limited control over custom logic like sockets/snap rules.

Conclusion: Use Three.js now for the Asset Lab and placement sandbox. Keep the option open to port the core viewer to a native app.

Planned Architecture (Asset Lab)

Core pieces we'll build with Three.js:

- Scene/Renderer: black background, ACES tone mapping, neutral environment lighting.
- Camera/Controls: Perspective camera with OrbitControls for rotate/pan/zoom.
- Loaders: GLTFLoader (GLB), KTX2Loader for compressed textures, Draco/Meshopt decoders.
- HUD/UI: HTML overlay for metrics (triangles, draw calls, textures, size) and actions (wireframe, exposure, export).
- Manifest + Local mode: load assets from drag-and-drop or a hosted manifest.json.
- Sockets/Placement: visualize shelf snap points and place objects (books, lamps) with fit checks.
- Save/Load Layouts: serialize placement JSON for later gameplay.
- Screenshots/Turntables: export PNG thumbnails; optional scripted rotation for turntable GIF/MP4.

Object Model & Separation of Concerns

OOP plan (portable to C#/GDScript later):

- Socket: id, pos, normal, size, tags, occupied/placements.
- Shelf: id, name, sockets[], query helpers (getFreeSockets).
- Placeable: id, name, footprint(w,d), baseHeight, rotationSnapDeg, allowedSockets.
- PlacementManager: rules (canFit, snapRotation), reserve/release, serialize/load layout.

This separation keeps rules centralized and models simple, lowering complexity as you add content.

Performance & Asset Guidelines

Performance guidelines:

- Use optimized Runtime assets (GLB + KTX2 textures). Avoid raw PNGs/JPGs in the viewer.
- Target prop budgets: 1–5k triangles typical; shelves can be higher but keep LODs.

Performance & Asset Guidelines (cont.)

- Share materials: use atlases/trim sheets to minimize draw calls.
- Use Draco or Meshopt compression; prefer ASTC/BCn via KTX2 for textures.
- Lazy-load: load one asset at a time; dispose geometries/materials when switching.
- Mobile: cap texture dimensions (e.g., ≤ 2048) and test on mid-range devices.

Pipeline Integration (Runtime & CI)

Pipeline integration:

- Keep authoring in Blender; export GLB (meters, transforms applied), sidecar JSON for metadata.
- Run `asset_drop.sh` to bake: creates `/Runtime/<type>/<id>` with `models/`, `metadata/`, `textures/`.
- Three.js viewer reads from `/Runtime` locally or via a hosted `manifest.json`.
- CI builds `manifest.json` automatically and publishes viewer + assets to GitHub Pages.
- The same viewer doubles as a QA tool: flags oversized assets and bad pivots early.

Security, Hosting, and Access

Security & deployment:

- Host on GitHub Pages (private while testing). Flip to public for demo later.
- Only publish baked Runtime assets (optimized). Keep source `.blend/` `.fbx` private.
- Use Git LFS for large source files (if they ever enter the repo).
- Version manifests; consider cache-busting (query strings) for frequent updates.

Risks & Mitigations

Risks & mitigations:

- Browser limitations (file system access): solved via drag-and-drop and hosted manifests.
- Heavy assets causing slow loads: stick to Runtime outputs, enforce budgets in CI.
- Feature creep: keep the viewer modular (Asset mode vs Shelf/Placement mode).
- Future engine switch: keep placement logic in small, pure classes to port later.

Roadmap (Next 2–4 Weeks)

Roadmap:

Week 1: finalize viewer (theme, manifest auto-load), add HUD, export PNG, size badges.

Week 2: Shelf Mode: render sockets, drag-and-drop placeables, fit checks, save/load layouts.

Week 3: Batch thumbnails/turntables, public demo polish, soft-launch Pages site.

Week 4+: Expand libraries (books, lamps, plants), add basic scoring and stack physics preview.

Code Skeleton (High Level)

Minimal starter (pseudocode):

```
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(50, w/h, 0.01, 1000);
const renderer = new THREE.WebGLRenderer({antialias:true});
renderer.setSize(w,h); document.body.appendChild(renderer.domElement);
const loader = new GLTFLoader(); loader.load('Runtime/.../model.glb', gltf => scene.add(gltf.scene));
function animate(){ requestAnimationFrame(animate); renderer.render(scene,camera); } animate();
```

Next steps in code: add OrbitControls, lights, HUD overlay, and the socket/placement classes.

Next Steps

Action items:

- 1) Keep using the themed viewer (already built) as the Asset Lab base.
- 2) Add manifest generation to CI so the website lists assets automatically.
- 3) Implement Shelf Mode (sockets visualized) with the OOP classes above.
- 4) Start with one shelf + 5–10 placeables to validate snapping and layout save/load.
- 5) Review performance on desktop + one mid-range phone; adjust budgets.